Software Requirements Specification (SRS)

ClarifyFinance: Human-Centered Personal & Group Expense Orchestrator

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the complete functional, behavioral, and non-functional requirements for **ClarifyFinance**, an AI-augmented personal cash flow and collaborative group expense orchestrator. Designed around Human-Computer Interaction (HCI) and Human-Centered Design (HCD) principles, this document provides the foundation for engineering, validation, and academic defense.

1.2 Product Scope

ClarifyFinance unifies individual expense tracking with multi-party debt settlement into a cohesive, multimodal web platform. It eliminates manual expense-logging fatigue, simplifies itemized bill distribution, and replaces "black-box" debt calculations with an explainable settlement topology. The platform runs entirely on reproducible, open-source models and self-hosted infrastructure.

1.3 Target Audience & User Personas

- **The Social Trip Organizer:** Frequently manages shared dining, travel, and household expenses. Values rapid itemized bill entry, automated tax/tip distribution, and zero social friction when requesting payments.
- **The Passive Group Member:** Needs complete transparency into why debts are routed a certain way without having to audit receipts or calculate spreadsheets manually.
- **The Anxious Budgeter:** Wants clear financial pacing without facing guilt-inducing red alerts or complex manual ledger tracking.

2. Overall Description

2.1 System Perspective

ClarifyFinance operates as a modular full-stack web application. The client interface handles direct manipulation, speech recording, and graph rendering. The application layer orchestrates the relational database, local AI processing pipelines, and debt-simplification algorithms.

2.2 Operating Environment

- **Client Environment:** Modern standards-compliant web browsers supporting Web Audio, WebGL, and touch/mouse input.
- **Server Environment:** POSIX-compliant operating system running Python and Node.js environments with an embedded local relational database engine.

2.3 General Constraints

- **Open-Source Footprint:** All application frameworks, AI models, and database engines must be freely available and reproducible on local hardware without paid enterprise APIs.
- **Human-Centered Principle of System Status:** AI outputs (transcripts, OCR scans, parsed items) must always reside in a staging state for user approval before anything is committed to the financial ledger.

3. Functional Requirements

3.1 Multimodal Expense Logging

- **Voice Ingestion:** The system captures spoken expense descriptions in natural language, processes the audio stream locally, and outputs normalized text.
- **Natural Language Entity Extraction:** The system parses conversational sentences into structured entities including payer identity, total amount, currency, participating members, and item splits.
- **Interactive Staging Chips:** Extracted entities are presented as editable visual chips. Users can tap or click any chip to alter values before confirming ledger entry.

3.2 Receipt OCR & Itemization Canvas

- **Document Processing:** The system processes uploaded receipt images, normalizes contrast, and extracts text lines alongside their visual spatial positions.
- **Itemized Extraction:** Line items are extracted into item descriptions, unit counts, individual prices, total tax, and gratuity amounts.

- **Tap-to-Claim Canvas:** Line items are rendered as modular visual cards. Users tap or drag participant avatars onto specific items to assign ownership.
- **Proportional Surcharge Distribution:** The system automatically distributes taxes, tips, and discounts proportionally across participants based on their share of the base subtotal.

3.3 Explainable Group Debt Engine

- **Net Balance Computation:** The system computes the net financial position of each group member by subtracting total owed balances from total paid outlays.
- **Debt Simplification:** The system reduces the total number of transactions needed across the group using a bipartite balance reduction method, settling N members in at most $N-1$ payments.
- **Transitive Proof Generation:** The engine maintains historical dependencies to produce plain-language justifications for routed debts (e.g., explaining why Member A pays Member C directly instead of Member B).
- **Interactive Graph Visualization:** The final settlement topology is rendered as an interactive node graph, allowing users to select any payment edge to view its full audit history.

3.4 Personal Cash Flow & "Safe-to-Spend" Metric

- **Ledger Segregation:** Group expenses paid upfront by a user are partitioned so that only the user's personal share affects their monthly budget, while the remaining balance is tracked as a pending group receivable.
- **Safe-to-Spend Computation:** The system calculates immediately available discretionary funds by projecting upcoming recurring charges and adjusting for pending group repayments.
- **Non-Punitive Visual Feedback:** Budget pacing is displayed using neutral progress bands and buffer gauges rather than alarming alert colors.

3.5 Social Settlement Integrations

- **Direct Payment Links:** Settled debt edges provide one-click deep links for payment interfaces with pre-filled payee IDs and amounts.
- **Neutral Reminder Generation:** The system drafts objective, polite payment summaries that users can copy directly into group chats to remove social discomfort.

4. Non-Functional Requirements

4.1 HCI & Usability (Nielsen Heuristics & HCD)

- **Visibility of System Status:** All background AI processing states (speech inference, OCR parsing) must present visual activity indicators within 100 milliseconds of user input.
- **User Control and Freedom:** Every ledger modification and split assignment must provide an accessible, non-destructive Undo window lasting 10 seconds.
- **Ergonomics & Touch Affordances:** Primary interaction elements (recording buttons, claim avatars, confirmation buttons) must meet a minimum target size of 48 by 48 pixels and sit within natural screen-reach zones.
- **Accessibility Standards:** Text elements must adhere to WCAG 2.1 AA guidelines with a minimum visual contrast ratio of 4.5:1. Interface status must never rely on color alone.

4.2 Performance Requirements

- **Audio Inference Latency:** Spoken inputs under 15 seconds must be transcribed and structured within 2.5 seconds on consumer-grade quad-core CPUs.
- **Graph Calculation Latency:** Group debt simplification and transitive proof generation for up to 50 members must complete in under 50 milliseconds.
- **Visual Frame Rates:** The interactive settlement canvas must sustain a smooth 60 frames per second during drag, pan, and zoom interactions.

4.3 Privacy & Data Governance

- **Local Data Sovereignty:** Personal ledgers, receipt scans, and group records must reside on local, user-managed storage.
- **Transient Audio Processing:** Audio streams recorded for transcription must be processed entirely in volatile memory and discarded immediately upon text extraction.

5. Verification & Academic Evaluation Framework

Metric ID	Parameter	Evaluation Protocol	Success Benchmark
VM-1	System Usability Scale (SUS)	10-question standardized post-test survey	Score ≥80.0 (Grade A)
VM-2	Bill Allocation Efficiency	Timed task: 5 members, 10 itemized items	Completion time ≤60 s
VM-3	Algorithmic Transparency Rating	7-point Likert scale on settlement clarity	Mean score ≥5.8/7.0

VM-4	Cognitive Workload Reduction	NASA-TLX comparison against manual split apps	≥30% drop in Mental Demand